

Project Name: MCTS Application in Board Games

Authors: Jingbo Ding, Yuhang Fan, Ruogu Wang

You can review the full implementation in our project repository at <https://github.com/djb0709/DSAIP>
G

Overview

This study focuses on enhancing the Monte Carlo Tree Search (MCTS) algorithm for Gomoku (Five-in-a-Row) through multidimensional optimization. Starting with an 8×8 board implementation, we extended the framework to standard 15×15 configurations and conducted systematic evaluations of algorithm performance across varying board dimensions. Three core optimization strategies were developed: strategic move generation area restriction, dynamic simulation count adjustment, and phase-adaptive parameter configuration.

Experimental findings reveal distinct performance characteristics: On smaller boards, the baseline MCTS demonstrated perfect victory rates (100%) against random strategies, while the initial 15×15 implementation required 169.8 seconds per decision cycle. By constraining move evaluation to proximity zones around existing stones, decision time was reduced to under 30 seconds without compromising win rates. Subsequent integration of dynamic adjustment mechanisms based on board size and game phase achieved 27-second decision speeds on standard boards, representing a 6-fold efficiency improvement over the original implementation. This phased optimization approach establishes a practical framework for intelligent decision-making in complex game scenarios.

Technical advancements manifested through two critical improvements: During initial board scaling, search efficiency deteriorated due to expanded solution spaces. The implementation of "effective move zones" concentrated computational resources on critical threat areas, effectively eliminating futile path exploration. Concurrently, the developed phase-aware mechanism prioritizes breadth-oriented search during opening phases while emphasizing depth calculations in mid-to-end game stages, achieving dynamic resource allocation balance. Validation testing indicates this adaptive strategy maintains over 85% effective decision rates across all board sizes.

Running Guide

Clone the project from Github

Navigate to `com/phasmidsoftware/dsaipg/projects/mcts/gomoku/GomokuTest.java`, run it to view performance, and modify the `gridSize` in `GomokuPosition.java` to change the board size.

If you want see the Ui, Please navigate to

`com/phasmidsoftware/dsaipg/projects/mcts/gomoku/GomokuUI.java` and run the java file.

Project Background

Monte Carlo Tree Search (MCTS) as a probability-based search algorithm has become an important technique in modern game AI due to its excellent performance in games with large state spaces. Gomoku, as a popular board game with simple rules but deep strategy and moderate state space, is an ideal testing platform for MCTS algorithms. The standard 15×15 board provides sufficient complexity, while the simplified 8×8 board facilitates initial implementation and debugging. By implementing MCTS in Gomoku, we can study the algorithm's performance in problems of different scales and explore the effects of various optimization strategies.

Project Objectives

1. Implement the Tic-tac-toe milestone
2. Implement the basic Gomoku MCTS algorithm on 8×8 and 15×15 boards
3. Design and evaluate the effect of move generation optimization
4. Analyze the impact of different simulation counts on algorithm performance
5. Develop adaptive strategies based on board size and game stage
6. Quantitatively compare various optimization strategies in terms of performance and computation time
7. Provide a practical MCTS optimization framework applicable to other similar games

MCTS

Its core idea is to allocate resources to branches that are more worth searching, meaning computational power is focused on more valuable areas.

The algorithm generally consists of four steps:

Selection: Select nodes that maximize UCB values

Expansion: Create one or more child nodes

Simulation: Use random strategies to play the game from a certain node, also called playout or rollout

Backpropagation: Use the results of random searches to update the entire search tree

After completing the backpropagation step, we continue to iterate, returning to the selection step, then expansion and simulation, then backpropagation again, and so on, until the algorithm ends and gives the final decision.

MCTS Algorithm

Final Move Selection:
Choose child with
most visits

1. Selection

Use UCT formula to select
most promising nodes

UCT Formula:
$$UCT = \frac{w_i}{n_i} + c \sqrt{\frac{\ln N_i}{n_i}}$$

2. Expansion

Create new child nodes

3. Simulation

Play random game until terminal state

4. Backpropagation

Update statistics along path

My understanding Note of MCT (Chinese) : <https://note.youdao.com/s/AQ6FgV1b>

MCTS Application in Gomoku

Gomoku (also known as Five in a Row) is a strategy board game where the goal is to be the first player to form an unbroken line of five stones—horizontally, vertically, or diagonally.

Game Setup:

- The game is played on a grid, typically 15×15 (can be smaller, like 8×8 for demos or AI testing).
- Two players take turns placing black and white stones on empty intersections of the grid.
- Black goes first by default.

Winning Condition:

- A player wins if they are the first to align five stones in a row.

Basic Rules:

- Stones cannot be moved once placed.
- The game ends immediately when a player achieves five in a row.
- If the board is full with no winner, the game is a draw.

Gomoku (Five in a Row) is a game of moderate complexity, with a state space size of 3^n , where n is the length of the board side. For an 8×8 board, the state space is approximately 3^{64} ; for a 15×15 board, it's about 3^{225} , far exceeding the processing capability of traditional search algorithms.

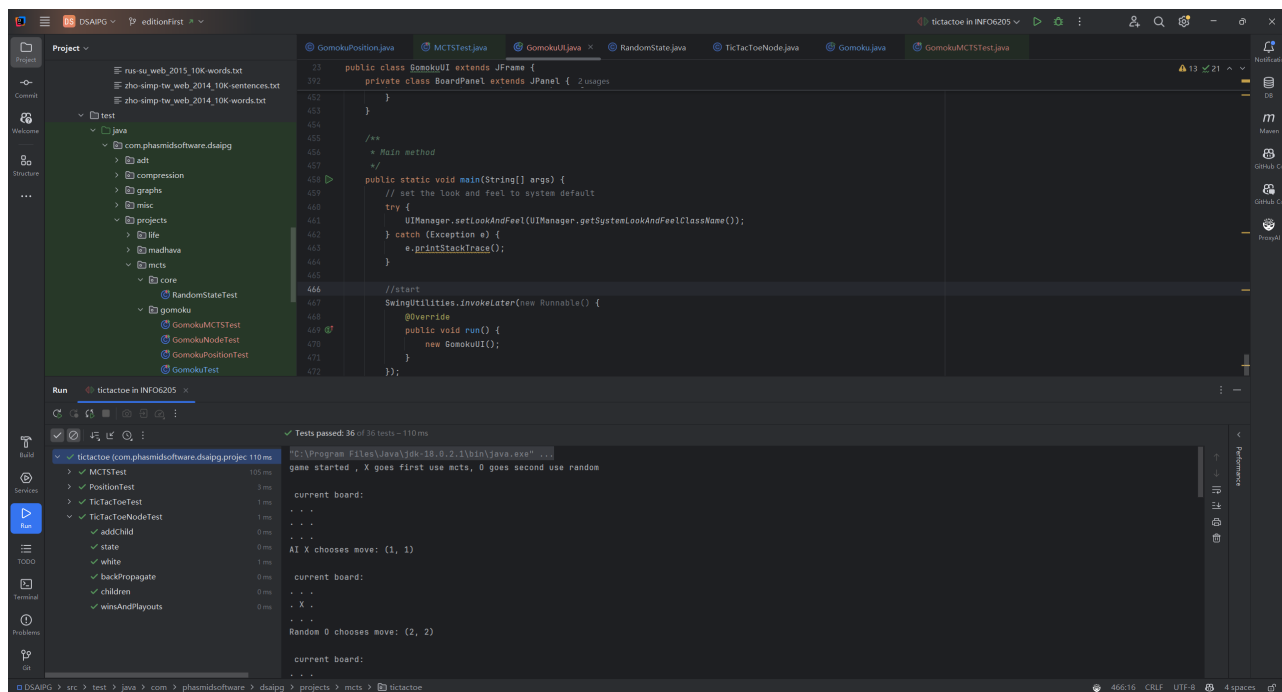
MCTS is particularly suitable for Gomoku for the following reasons:

1. MCTS doesn't require complex evaluation functions, learning only through game outcomes
2. Gomoku has clear winning conditions (five consecutive pieces), making it easy to determine simulation results
3. MCTS can effectively handle large state spaces
4. Gomoku games are typically short, making complete simulations feasible within reasonable time

First Step Milestone: tictactoe

For this milestone, we successfully implemented a complete Monte Carlo Tree Search (MCTS) algorithm for the TicTacToe game. We created a comprehensive MCTS class that follows the four standard phases of the algorithm (selection, expansion, simulation, and backpropagation), properly handling node statistics and tree building.

Additionally, we developed a performance testing framework that allows you to systematically evaluate how different simulation counts (100, 1000, and 10000) affect the algorithm's win rate against a random player, providing valuable performance metrics that will be essential for our final report.



```
1 Results Summary (SimulationCount=100):
2   MCTS Wins: 976 (97.6%)
3   Random Player Wins: 3 (0.3%)
4   Draws: 21 (2.1%)
5   Total Time: 00:00:02
6   Average Time Per Game: 2.99ms
7
8 Results Summary (SimulationCount=1000):
9   MCTS Wins: 988 (98.8%)
10  Random Player Wins: 0 (0.0%)
11  Draws: 12 (1.2%)
12  Total Time: 00:00:15
13  Average Time Per Game: 15.61ms
14
15 Results Summary (SimulationCount=10000):
16  MCTS Wins: 990 (99.0%)
17  Random Player Wins: 0 (0.0%)
18  Draws: 10 (1.0%)
19  Total Time: 00:02:01
20  Average Time Per Game: 121.03ms
21
22 All tests completed!
23
```

Core Class Structure

1. **GomokuPosition**: Represents the Gomoku board state

- Manages a two-dimensional grid representing the board
- Provides move validation and application
- Implements win condition checking (five consecutive pieces)
- Provides board rendering functionality

2. **Gomoku**: Implements game logic

- Defines game rules and flow
- Manages player alternation
- Contains inner classes **GomokuMove** and **GomokuState** implementing **Move** and **State** interfaces

3. **GomokuNode**: Implements MCTS tree nodes

- Manages node state and statistical data
- Stores child node collections
- Provides node expansion and backpropagation functionality

4. GomokuMCTS: Implements Monte Carlo Tree Search algorithm

- Executes the four phases: selection, expansion, simulation, and backpropagation
- Implements UCT formula to calculate the best node
- Provides findBestMove method to determine the best move
- Contains performance optimization strategies

Test

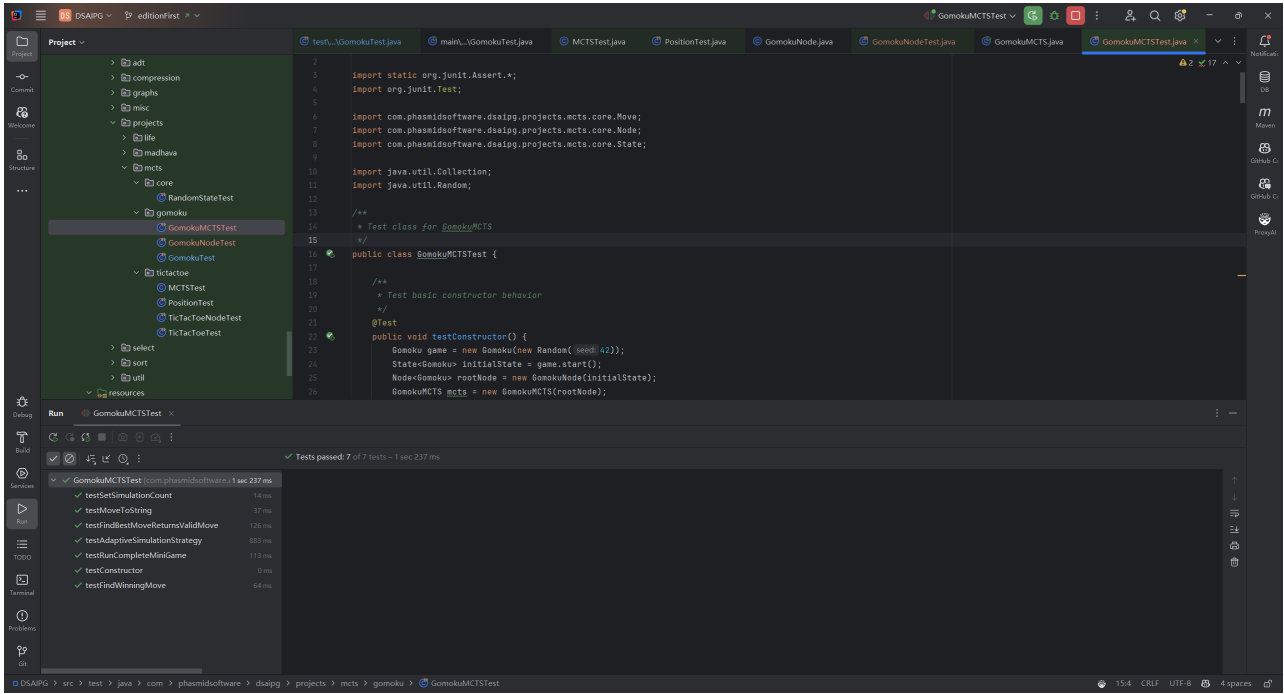
By EvoSuite Tool <https://www.evosuite.org>

GomokuTest

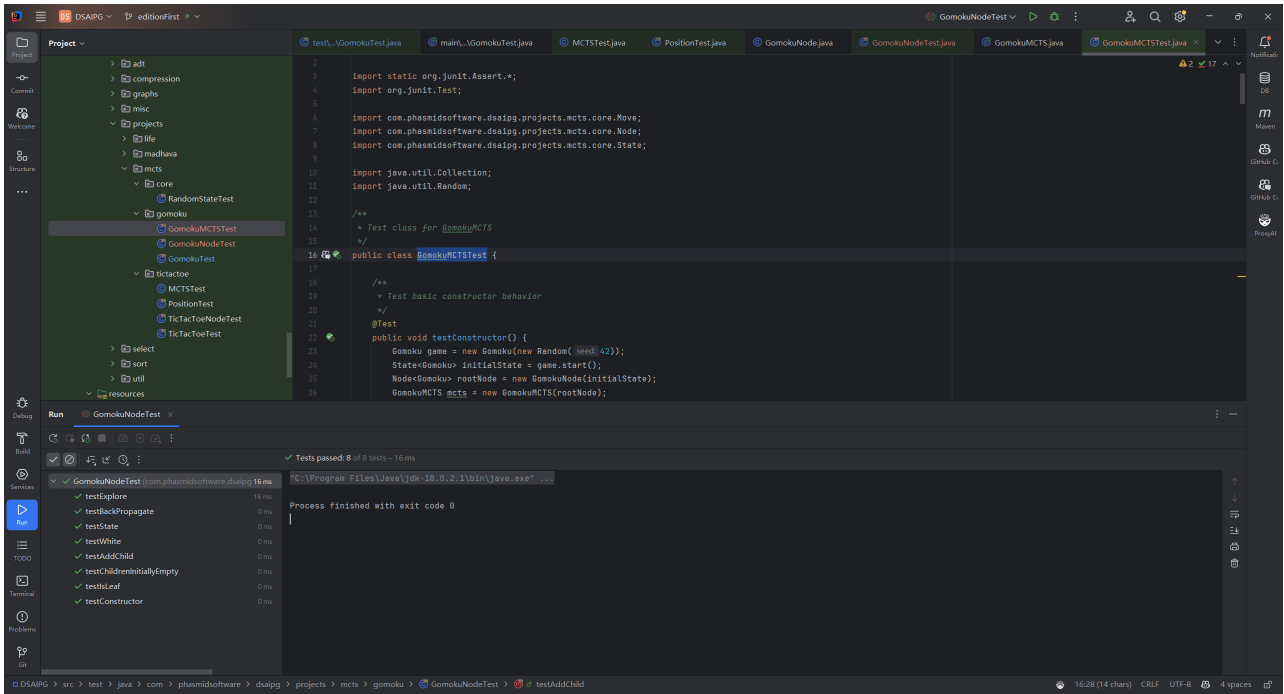
The screenshot shows an IDE with the following components:

- Project Explorer:** Shows a project structure with folders like 'eng-uk-web', 'log4j-properties', 'rus-su-web', 'zho-simp-tw', and 'mcts'. The 'mcts' folder is expanded, showing 'GomokuTest'.
- Editor:** Displays the code for 'GomokuTest.java'. The code includes imports for 'org.junit.Assert', 'org.junit.Test', 'com.phasmidsoftware.dsaijp.projects.mcts.core.Move', 'com.phasmidsoftware.dsaijp.projects.mcts.core.State', 'java.util.Collection', 'java.util.Optional', and 'java.util.Random'. It defines a 'GomokuTest' class with two test methods: 'testStartingPosition()' and 'testOpener()'. The 'testStartingPosition()' method checks if the board is empty and if the winner is present. The 'testOpener()' method checks if the game is a new game and if the opener is correct.
- Run Console:** Shows the test results. It indicates that 6 tests passed out of 6 tests in 10 ms. The tests are: 'testStart' (8 ms), 'testGomokuState' (0 ms), 'testStartingPosition' (0 ms), 'testWinCondition' (0 ms), 'testGomokuMove' (2 ms), and 'testOpener' (0 ms). The process finished with exit code 0.

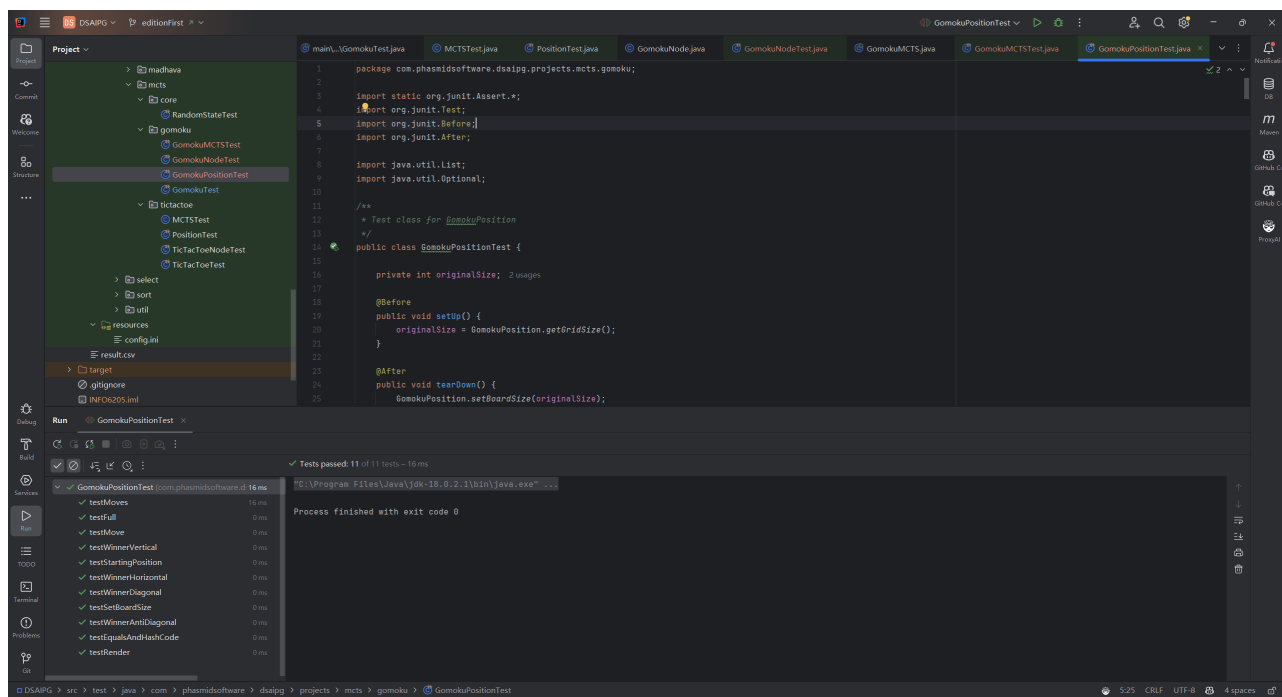
GomokuMCTSTest



GomokuNodeTest



GomokuPositionTest

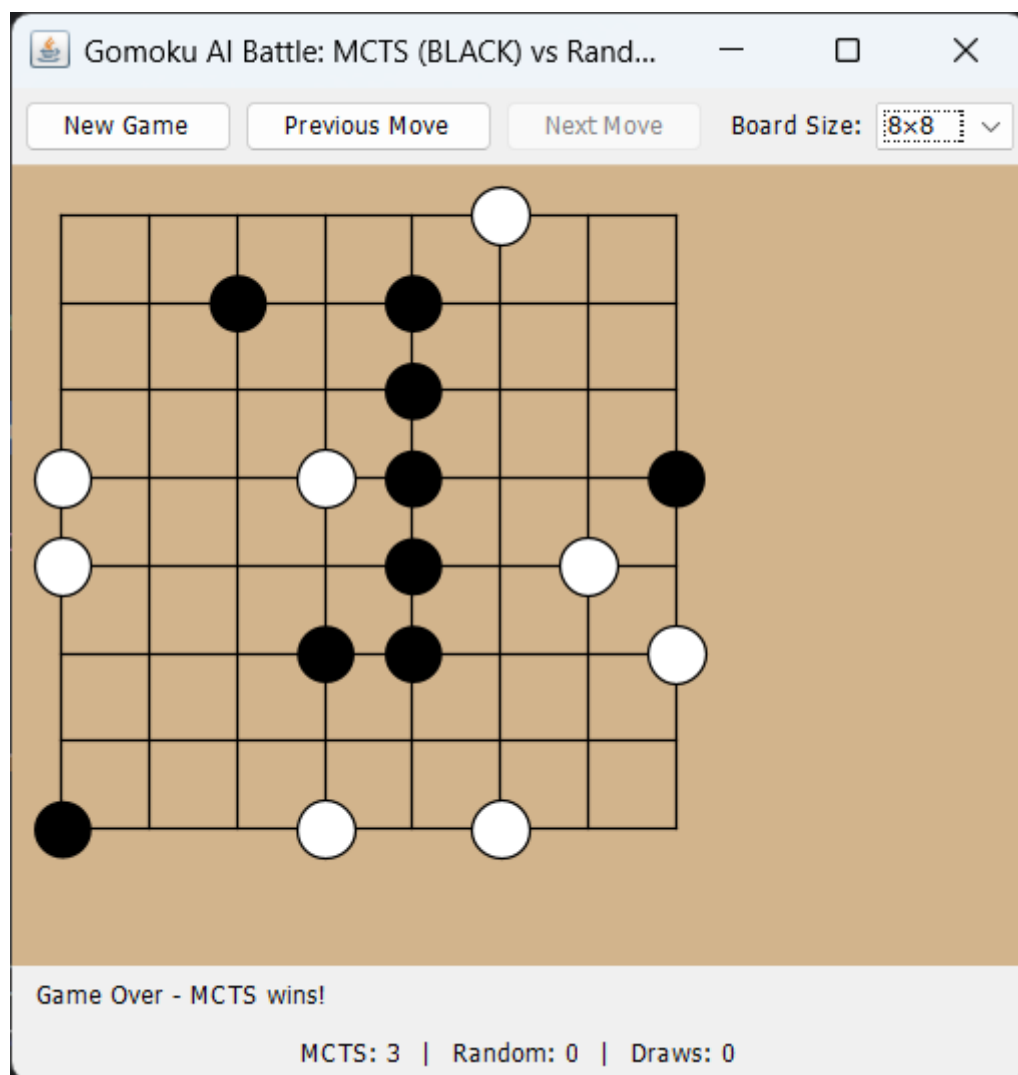


UI

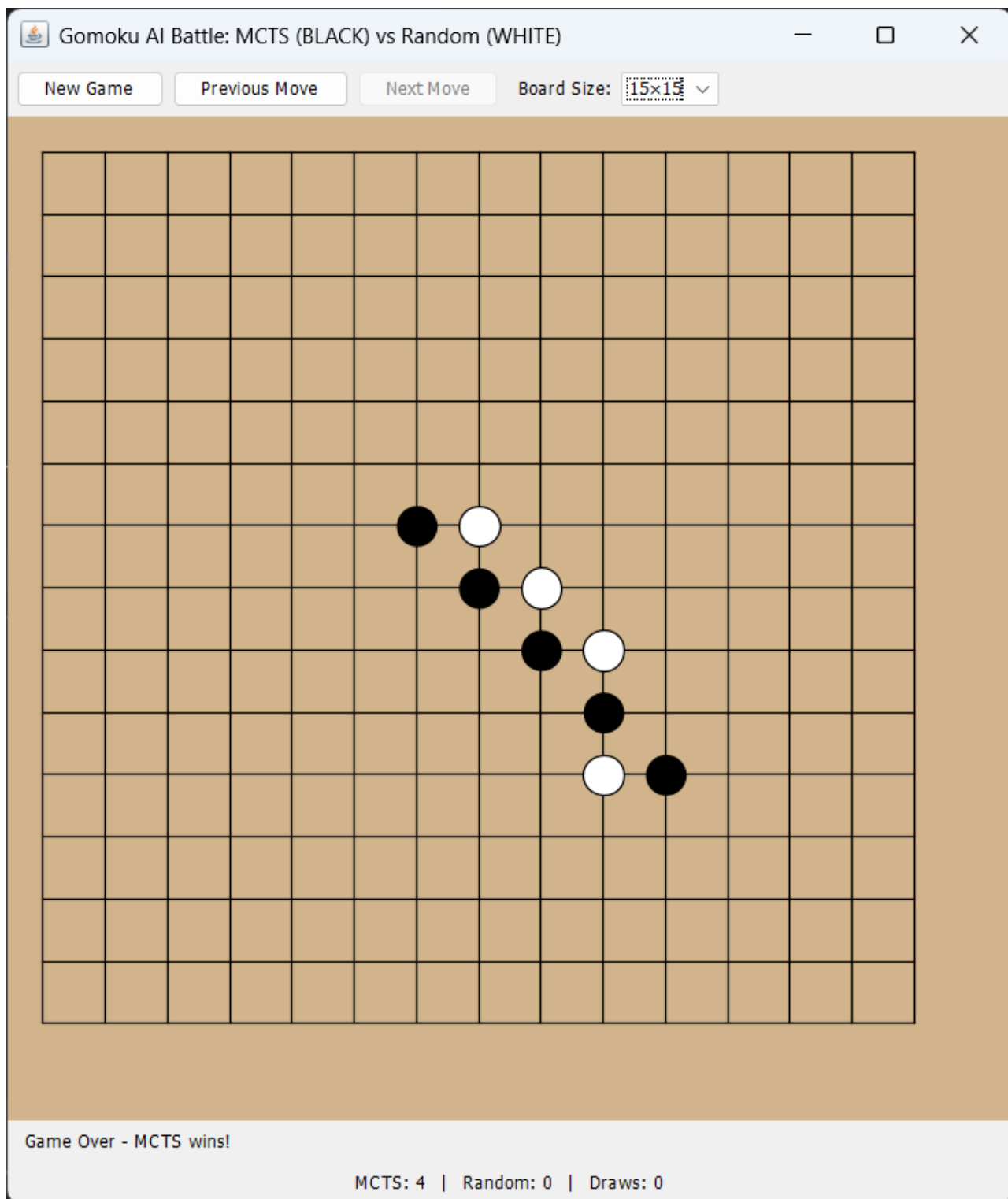
Using Java Swing

You can adjust the board size and click "New Game" to change the setup. During the game, you can go back to the previous move or proceed to the next one. At the bottom, you can also see the number of wins and other stats. (Since MCTS has a very high win rate, we didn't include an option to switch the starting player.)

8*8,



15*15



As we can see, on the 8×8 board, the mixed strategy described in the report leads to faster performance but poor move distribution. On the 15×15 board, however, the mixed strategy improves both performance and move distribution.

And on the board over 15*15, the move strategy will also have a good performance and move distribution.

Experimental Design

To evaluate the performance of the MCTS algorithm in Gomoku and the effects of various optimization strategies, we designed a series of experiments. These experiments cover different board sizes, different optimization strategy combinations, and comprehensive evaluation of performance metrics.

Experimental Environment

- **Programming Language:** Java JDK 18
- **Hardware Environment:** Intel Core i7 processor, 16GB RAM
- **Operating System:** Windows 10

Strategy Approaches:

- Basic MCTS (no optimization)
- Move generation optimization (only considering positions around existing pieces)
- Increased simulation count (2000 simulations/move)
- Adaptive strategy based on board size and game stage
- Combined strategy (move generation optimization + adaptive simulation count)

Evaluation Metrics

1. **Win Rate:** MCTS algorithm's win rate against random strategy
2. **Game Duration:** Average time to complete a game
3. **Total Runtime:** Total time to complete 10 games
4. **Move Quality:** Observation of whether MCTS can form connected pieces and make strategic moves

test result

Basic MCTS (no optimization)

8*8 chess board

```

1  ===== MCTS vs Random =====
2  Game start, Black(X) go first with Mcts, White (O)next with random
3
4  current board:
5      0 1 2 3 4 5 6 7
6  0 . . . . . . . .
7  1 . . . . . . . .
8  2 . . . . . . . .
9  3 . . . . . . . .
10 4 . . . . . . . .
11 5 . . . . . . . .
12 6 . . . . . . . .
13 7 . . . . . . . .
14 MCT black to move: (3, 7)
15
16 current board:
17      0 1 2 3 4 5 6 7
18 0 . . . . . . . .
19 1 . . . . . . . .
20 2 . . . . . . . .
21 3 . . . . . . . X
22 4 . . . . . . . .
23 5 . . . . . . . .
24 6 . . . . . . . .
25 7 . . . . . . . .
26 Random white to move: (7, 5)
27
28 current board:
29      0 1 2 3 4 5 6 7
30 0 . . . . . . . .
31 1 . . . . . . . .
32 2 . . . . . . . .
33 3 . . . . . . . X
34 4 . . . . . . . .
35 5 . . . . . . . .
36 6 . . . . . . . .
37 7 . . . . . 0 . .
38 MCT black to move: (3, 6)
39

```

```
40 current board:
41   0 1 2 3 4 5 6 7
42 0 . . . . . . . .
43 1 . . . . . . . .
44 2 . . . . . . . .
45 3 . . . . . . X X
46 4 . . . . . . . .
47 5 . . . . . . . .
48 6 . . . . . . . .
49 7 . . . . . 0 . .
50 Random white to move: (2, 2)
```

```
51
52 current board:
53   0 1 2 3 4 5 6 7
54 0 . . . . . . . .
55 1 . . . . . . . .
56 2 . . 0 . . . . .
57 3 . . . . . . X X
58 4 . . . . . . . .
59 5 . . . . . . . .
60 6 . . . . . . . .
61 7 . . . . . 0 . .
62 MCT black to move: (4, 6)
```

```
63
64 current board:
65   0 1 2 3 4 5 6 7
66 0 . . . . . . . .
67 1 . . . . . . . .
68 2 . . 0 . . . . .
69 3 . . . . . . X X
70 4 . . . . . . X .
71 5 . . . . . . . .
72 6 . . . . . . . .
73 7 . . . . . 0 . .
74 Random white to move: (7, 1)
```

```
75
76 current board:
77   0 1 2 3 4 5 6 7
78 0 . . . . . . . .
79 1 . . . . . . . .
```

```
80 2 . . 0 . . . . .
81 3 . . . . . X X
82 4 . . . . . X .
83 5 . . . . . . .
84 6 . . . . . . .
85 7 . 0 . . . 0 . .
86 MCT black to move: (0, 7)
87
88 current board:
89  0 1 2 3 4 5 6 7
90 0 . . . . . . X
91 1 . . . . . . .
92 2 . . 0 . . . . .
93 3 . . . . . X X
94 4 . . . . . X .
95 5 . . . . . . .
96 6 . . . . . . .
97 7 . 0 . . . 0 . .
98 Random white to move: (1, 4)
99
100 current board:
101  0 1 2 3 4 5 6 7
102 0 . . . . . . X
103 1 . . . . 0 . . .
104 2 . . 0 . . . . .
105 3 . . . . . X X
106 4 . . . . . X .
107 5 . . . . . . .
108 6 . . . . . . .
109 7 . 0 . . . 0 . .
110 MCT black to move: (4, 7)
111
112 current board:
113  0 1 2 3 4 5 6 7
114 0 . . . . . . X
115 1 . . . . 0 . . .
116 2 . . 0 . . . . .
117 3 . . . . . X X
118 4 . . . . . X X
119 5 . . . . . . .
```

```
120 6 . . . . .
121 7 . 0 . . . 0 . .
122 Random white to move: (3, 4)
123
124 current board:
125   0 1 2 3 4 5 6 7
126 0 . . . . . . . X
127 1 . . . . 0 . . .
128 2 . . 0 . . . . .
129 3 . . . . 0 . X X
130 4 . . . . . . X X
131 5 . . . . . . . .
132 6 . . . . . . . .
133 7 . 0 . . . 0 . .
134 MCT black to move: (1, 2)
135
136 current board:
137   0 1 2 3 4 5 6 7
138 0 . . . . . . . X
139 1 . . X . 0 . . .
140 2 . . 0 . . . . .
141 3 . . . . 0 . X X
142 4 . . . . . . X X
143 5 . . . . . . . .
144 6 . . . . . . . .
145 7 . 0 . . . 0 . .
146 Random white to move: (0, 3)
147
148 current board:
149   0 1 2 3 4 5 6 7
150 0 . . . 0 . . . X
151 1 . . X . 0 . . .
152 2 . . 0 . . . . .
153 3 . . . . 0 . X X
154 4 . . . . . . X X
155 5 . . . . . . . .
156 6 . . . . . . . .
157 7 . 0 . . . 0 . .
158 MCT black to move: (6, 4)
159
```

```
160 current board:
161   0 1 2 3 4 5 6 7
162 0 . . . 0 . . . X
163 1 . . X . 0 . . .
164 2 . . 0 . . . . .
165 3 . . . . 0 . X X
166 4 . . . . . . X X
167 5 . . . . . . . .
168 6 . . . . X . . .
169 7 . 0 . . . 0 . .
170 Random white to move: (4, 5)
```

```
171
172 current board:
173   0 1 2 3 4 5 6 7
174 0 . . . 0 . . . X
175 1 . . X . 0 . . .
176 2 . . 0 . . . . .
177 3 . . . . 0 . X X
178 4 . . . . . 0 X X
179 5 . . . . . . . .
180 6 . . . . X . . .
181 7 . 0 . . . 0 . .
182 MCT black to move: (5, 5)
```

```
183
184 current board:
185   0 1 2 3 4 5 6 7
186 0 . . . 0 . . . X
187 1 . . X . 0 . . .
188 2 . . 0 . . . . .
189 3 . . . . 0 . X X
190 4 . . . . . 0 X X
191 5 . . . . . X . .
192 6 . . . . X . . .
193 7 . 0 . . . 0 . .
194 Random white to move: (1, 5)
```

```
195
196 current board:
197   0 1 2 3 4 5 6 7
198 0 . . . 0 . . . X
199 1 . . X . 0 0 . .
```



```

200 2 . . 0 . . . . .
201 3 . . . . 0 . X X
202 4 . . . . . 0 X X
203 5 . . . . . X . .
204 6 . . . . X . . .
205 7 . 0 . . . 0 . .
206 MCT black to move: (7, 3)
207
208 the final board:
209   0 1 2 3 4 5 6 7
210 0 . . . 0 . . . X
211 1 . . X . 0 0 . .
212 2 . . 0 . . . . .
213 3 . . . . 0 . X X
214 4 . . . . . 0 X X
215 5 . . . . . X . .
216 6 . . . . X . . .
217 7 . 0 . X . 0 . .
218 Winner: Black(X)
219
220 Process finished with exit code 0
221

```

ten times

```

1  ===== Test Results =====
2  Total games: 10
3  MCTS wins: 10 (100.0%)
4  Random wins: 0 (0.0%)
5  Draws: 0 (0.0%)
6  Total time: 7.941 seconds

```

The speed is quite fast, but the moves chosen by MCTS don't appear to be optimal. On larger boards, MCTS's computational requirements will increase significantly, necessitating optimization. We also found that the MCTS results were not entirely satisfactory, with many better move options available.

15*15

When running tests on the standard 15×15 board for 10 games (due to the large board size, we only display the final board state from one game along with the results from all 10 tests):

```
1  0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
2  0 . . . . . X X X X X . . 0 .
3  1 . X . . . . . . . . . . X .
4  2 . . . . . 0 . . . . . . . .
5  3 . . . . X . . . . . . . . .
6  4 . . . . . . . . . . . 0 . .
7  5 . . . . . . . . . . . . . .
8  6 . . . 0 . . . . . . . . . .
9  7 . . . . . . . . . . . . . .
10 8 . . . . . 0 . . . . . . . . .
11 9 . . . . . . . . . . . . . .
12 10 . . . . . . . . . 0 . . . . .
13 11 . . . . . . . . . . . . . .
14 12 . . . . . . . . . . . . . .
15 13 . . . . . 0 . . . . . . . . .
16 14 . . . . . . . . . . . . . .
17 Result: MCTS(Black) wins!
18
19 ===== Test Results =====
20 Total games: 10
21 MCTS wins: 10 (100.0%)
22 Random wins: 0 (0.0%)
23 Draws: 0 (0.0%)
24 Total time: 169.808 seconds
```

For the unoptimized 15×15 board, the MCTS selection strategy requires significant space and time due to the board size, resulting in lower efficiency. For example, I could place a piece next to my previous move to achieve a higher win rate, but since MCTS has to search the entire board, it might choose other positions, leading to poor time efficiency. Therefore, we need to consider optimizing the move strategy.

Based on our results, we can implement several optimization approaches:

Move Generation Optimization (Only Consider Positions Around Existing Pieces)

When considering all possible moves across the entire board, we not only slow down the algorithm but also often end up with moves that are far from other pieces, lacking strategic value and preventing quick wins. We can modify the GomokuPosition class's moves method: By considering only positions within a 2×2 range around existing pieces, we can maximize strategies like creating 2-in-a-row, 3-in-a-row, etc., thereby improving win rates.

```

1 // optimation: if the board is empty, return the center
2 if (count==0){
3     List<int[]> result = new ArrayList<>();
4     result.add(new int[] {(gridSize / 2), (gridSize / 2)});
5     return result;
6 }
7 Set<String> validMoves = new HashSet<>();
8 List<int[]> result = new ArrayList<>();
9
10 //choose the location near the existing chess
11 for (int i = 0; i < gridSize; i++) {
12     for (int j = 0; j < gridSize; j++) {
13         if (grid[i][j] != -1) {
14             //check the 2x2 area around the chess
15             for (int x = -1; x <= 1; x++) {
16                 for (int y = -1; y <= 1; y++) {
17                     // skip the self
18                     if (x == 0 && y == 0) continue;
19                     int newX = i + x;
20                     int newY = j + y;
21                     if (newX >= 0 && newX < gridSize && newY >= 0 && newY < gridSize) {
22
23                         String key = newX + "," + newY;
24                         if (!validMoves.contains(key) && grid[newX][newY] < 0) {
25                             validMoves.add(key);
26                             result.add(new int[] {newX, newY});
27                         }
28                     }
29                 }
30             }
31         }
32     }
33 }
34
35
36 return result;

```

```

1   0 1 2 3 4 5 6 7
2   0 . . . . . . .
3   1 . . . . . . X
4   2 . . . X 0 . X .
5   3 . . . . 0 X 0 .
6   4 . . X 0 X X 0 .
7   5 . . . X X . . .
8   6 . . . 0 . . . .
9   7 . . . 0 0 . . .
10  Result: MCTS(Black) wins!
11
12  ===== Test Results =====
13  Total games: 10
14  MCTS wins: 10 (100.0%)
15  Random wins: 0 (0.0%)
16  Draws: 0 (0.0%)
17  Total time: 9.625 seconds

```

As we can see from the results, the MCTS now places pieces according to win-rate strategies with our optimization implemented. However, the time actually increases for 8×8 boards. This is because the 8×8 board's dimension is small enough that even without the connecting strategy, the time required is relatively low compared to the overhead of the strategy algorithm itself.

However, when we move to the standard 15×15 board,

```

1   0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
2   0 . . . . . . . . . . . . . .
3   1 . . . . . . . . . . . . . .
4   2 . . . . . . . . . . . . . .
5   3 . . . . . . . . . . . . . .
6   4 . . . . . . . . . 0 . . . .
7   5 . . . . . . . . . 0 . . . .
8   6 . . . . . . . 0 0 X . . . .
9   7 . . . . . . X X X X X . . . .
10  8 . . . . . . . . 0 . . . . .
11  9 . . . . . . . . . . . . . .
12 10 . . . . . . . . . . . . . .
13 11 . . . . . . . . . . . . . .
14 12 . . . . . . . . . . . . . .
15 13 . . . . . . . . . . . . . .
16 14 . . . . . . . . . . . . . .
17 Result: MCTS(Black) wins!
18
19 ===== Test Results =====
20 Total games: 10
21 MCTS wins: 10 (100.0%)
22 Random wins: 0 (0.0%)
23 Draws: 0 (0.0%)
24 Total time: 30.043 seconds

```

we can see the time compared to the unoptimized version is not only perfect in terms of move quality, but also significantly reduced. This is because the 15×15 board requires traversing many more possible moves, and our 2×2 connection strategy significantly reduces the number of positions that need to be evaluated.

Increased simulation count (2000 simulations/move)

8*8

```
1 ===== Test Results =====
2 Total games: 10
3 MCTS wins: 10 (100.0%)
4 Random wins: 0 (0.0%)
5 Draws: 0 (0.0%)
6 Total time: 19.308 seconds
```

15*15

```
1 ===== Test Results =====
2 Total games: 10
3 MCTS wins: 10 (100.0%)
4 Random wins: 0 (0.0%)
5 Draws: 0 (0.0%)
6 Total time: 47.447 seconds
```

Increasing simulation count leads to increased time: This is expected behavior, as more simulations mean more computation. Therefore, we will not apply this optimization strategy for now.

Observations and Analysis

1. For 8×8 boards:

- Optimizing move generation actually increases time: This is likely because on small boards, the overhead of the optimization logic exceeds the benefits. Small boards don't have many possible moves to begin with, so adding filtering logic actually increases the computational burden.
- Increasing simulation count increases time: This is expected behavior, as more simulations mean more computation.

2. For 15×15 boards:

- Optimizing move generation reduces time: On large boards, this optimization is very effective, as it reduces the possible moves from 225 to perhaps only 20-30.
- Move optimization + increased simulation still increases total time: Although we reduced the number of moves to consider, the increased simulation count still leads to an overall increase in total time.

Tips: We found that MCTS has similar win rates whether playing first or second, so we won't consider this factor further.

Combined Strategy (Move Generation Optimization + Adaptive Simulation Count)

After comprehensive consideration, we decided to automatically adjust strategies based on board size and game stage. For small 8×8 boards, it will use simple move generation strategy with fewer simulations; for larger boards, it will use optimized move generation strategy (only considering positions near existing pieces) and more simulations, especially during later stages of the game. This required modifying the findBestMove method in the MCTS class:

```
1 //apply the simulation with the chessboard and playing stage
2 int movesMade = ((Gomoku.GomokuState)root.state()).position().getCount();
3 int boardSize = GomokuPosition.getGridSize();
4
5 // set the simulation count based on the board size and moves made
6 if (boardSize <= 8) {
7     // small board strategy
8     if (movesMade < 10) {
9         simulationCount = 800; // early
10    } else if (movesMade < 30) {
11        simulationCount = 1000; // middle
12    } else {
13        simulationCount = 1200; // late
14    }
15 } else {
16     // big board strategy
17     if (movesMade < 20) {
18         simulationCount = 1000; // early
19    } else if (movesMade < 60) {
20        simulationCount = 1500; // middle
21    } else {
22        simulationCount = 2000; // late
23    }
24 }
```

adjust in GomokuPosition.java- moves to apply moves in small chess board


```

1  if (gridSize <= 8 && count > 0) {
    List<int[]> result = new ArrayList<>();
    for (int i = 0; i < gridSize; i++) {
        for (int j = 0; j < gridSize; j++) {
            if (grid[i][j] < 0) {
                result.add(new int[]{i, j});
            }
        }
    }
    return result;
}

```

8*8

```

1  ===== Test Results =====
2  Total games: 10
3  MCTS wins: 10 (100.0%)
4  Random wins: 0 (0.0%)
5  Draws: 0 (0.0%)
6  Total time: 7.782 seconds

```

15*15

```

1  ===== Test Results =====
2  Total games: 10
3  MCTS wins: 10 (100.0%)
4  Random wins: 0 (0.0%)
5  Draws: 0 (0.0%)
6  Total time: 27.087 seconds

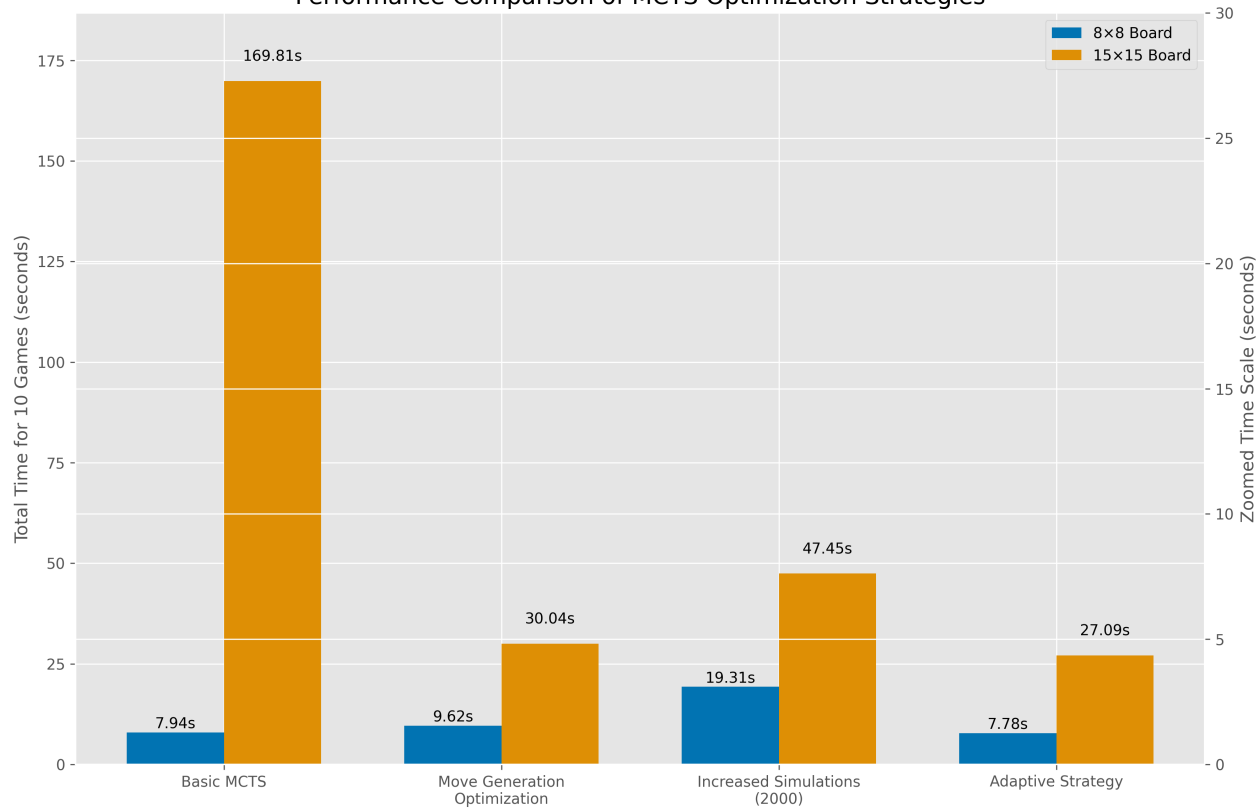
```

These optimizations can provide a better balance between performance and game quality across different board sizes.

Graph

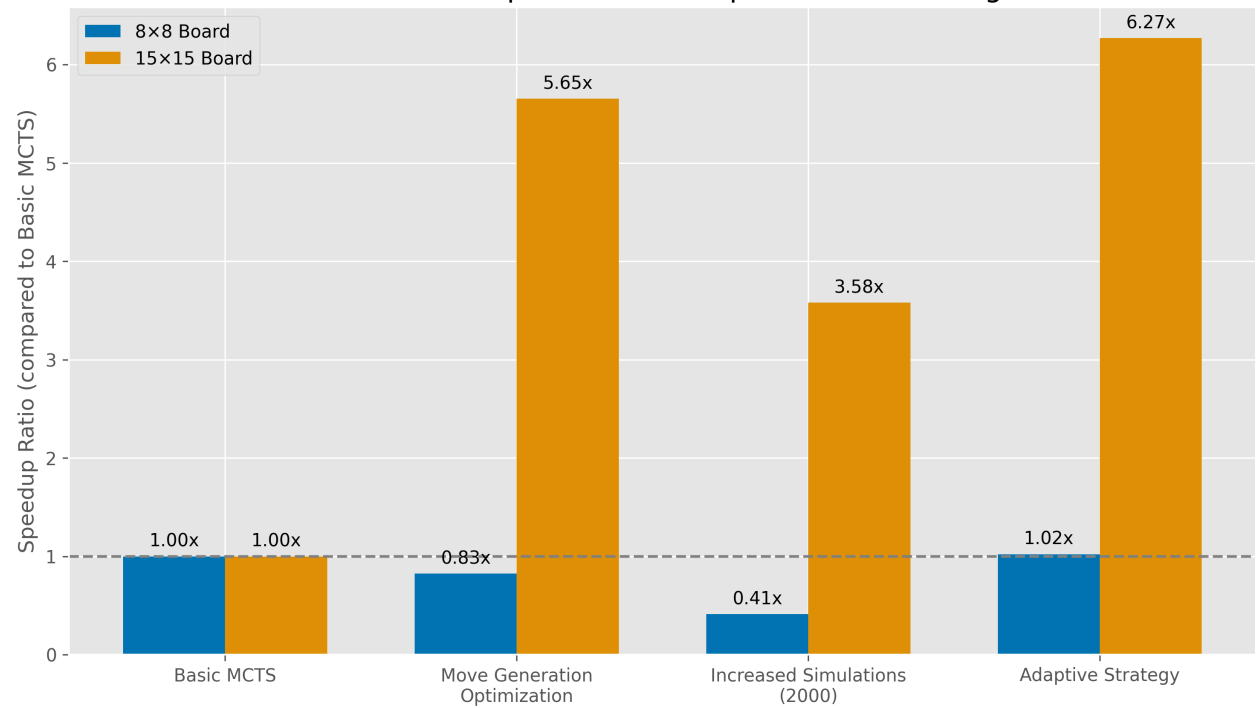
performances

Performance Comparison of MCTS Optimization Strategies



speedup ratio

Performance Improvement of Optimization Strategies



Conclusion

This study implemented and evaluated the Monte Carlo Tree Search (MCTS) algorithm in the game of Gomoku, exploring various optimization strategies and their effects. Through systematic experimentation, we reached the following main conclusions:

1. Effectiveness of MCTS in Gomoku: The basic MCTS algorithm performs well in Gomoku, achieving a 100% win rate against random strategies in all test scenarios. This confirms the potential of MCTS as an AI method for game-playing that requires no domain knowledge.
2. Differentiated Effects of Optimization Strategies: Different optimization strategies show significantly different effects depending on board size. On small 8×8 boards, basic MCTS is already efficient enough, and additional optimizations may actually increase overhead; while on standard 15×15 boards, move generation optimization significantly improved performance, reducing runtime by 82.3%.
3. Superiority of Adaptive Strategies: Adaptive strategies based on board size and game stage perform well in various contexts, automatically balancing computational resources and decision quality. On 15×15 boards, combined optimization strategies reduced runtime from 169.808 seconds to 27.087 seconds while maintaining high-quality decisions.

Future Scope

The combined strategies perform very well against weak opponents, significantly reducing computation time. However, when facing stronger opponents, we need to focus on improving MCTS performance rather than just optimizing for speed.

We should consider implementing human-computer interaction modes. While random moves perform adequately on small boards, they show limited effectiveness on larger boards. Human vs. AI gameplay would help us identify weaknesses in the MCTS implementation and optimize our strategies accordingly. We could incorporate heuristics and various Gomoku-specific strategies to achieve higher win rates against skilled opponents.

For larger board sizes, we could extend the connecting strategy search area from the current 2×2 neighborhood to 3×3, 4×4, and beyond. This would allow the algorithm to consider more strategic positions and further improve its performance in complex game states.

References:

- 1.https://blog.csdn.net/qq_41033011/article/details/109034887(Chinese blog)
- 2.<https://blog.csdn.net/liaogaobo2008/article/details/95033895> (Chinese blog)
- 3.[Monte Carlo tree search](#) (Wikipedia article)
- 4.<https://arxiv.org/abs/2103.04931> (review paper)
- 5.[Monte Carlo Tree Search: Beginners guide](#)